Python CW(11/8.26)

1.

8/14/26, 9:15 AM Student Submissions — Class Work (11.8.26)

SIMATS Engineering • CSE • CSA0804 — Python Programming • 14-08-2026 • Category: Class Work (11.8.26) • Student Submissions Report

Student 28 **Sivarangjenii B** 192521375RD 10 / 10 submitted

Q1. Selection Sort

Score: 10.00 **TC: 3/3 passed**

CODE

```
a = list(map(int, input().split()))

n = len(a)

for i in range(n - 1):
    min_index = i

    for j in range(i + 1, n):
        if a[j] < a[min_index]:
            min_index = j

    a[i], a[min_index] = a[min_index], a[i]

print(a)
```

INPUT `23 78 45 8 32 56`

OUTPUT `[8, 23, 32, 45, 56, 78]`

2.

Q2. Insertion Sort

Score: 10.00 **TC: 3/3 passed**

CODE

```
a = list(map(int, input().split()))

for i in range(1, len(a)):
    key = a[i]
    j = i - 1

    while j ≥ 0 and a[j] > key:
        a[j + 1] = a[j]
        j -= 1

    a[j + 1] = key

print(a)
```

INPUT `8 5 7 1 9 3`

OUTPUT `[1, 3, 5, 7, 8, 9]`

3.

SIMATS Engineering • CSE • CSA0804 — Python Pro

Q3. Merge Sort

Score: 10.00 **TC: 3/3 passed**

CODE

```
a = list(map(int, input().split()))

def merge_sort(a):
    if len(a) <= 1:
        return a

    mid = len(a) // 2
    left = merge_sort(a[:mid])
    right = merge_sort(a[mid:])

    result = []
    i = 0
    j = 0

    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1

    result.extend(left[i:])
    result.extend(right[j:])

    return result

a = merge_sort(a)
print(a)
```

3.

```
    return result

a = merge_sort(a)
print(a)
```

INPUT

```
38 27 43 3 9 82 10
```

OUTPUT

```
[3, 9, 10, 27, 38, 43, 82]
```

Q4. Diagonal Matrix

Score: 10.00 **TC: 3/3 passed**

CODE

```
s = input().strip()

s = s[2:-2]
rows = s.split("],[")

matrix = []

for row in rows:
    matrix.append(list(map(int, row.split(","))))

diagonal = True

for i in range(len(matrix)):
    for j in range(len(matrix)):
        if i != j and matrix[i][j] != 0:
            diagonal = False
            break

if diagonal:
    print("Diagonal Matrix")
else:
    print("Not Diagonal Matrix")
```

INPUT

```
[[1,0,0],[0,5,0],[0,0,8]]
```

OUTPUT

```
Diagonal Matrix
```

4.

Q5. Row and Column Sums

Score: 6.67 TC: 2/3 passed

CODE

```
s = input().strip()

matrix = []
row = []
num = ""

for ch in s:
    if ch.isdigit() or ch == '-':
        num += ch
    elif ch == ',' or ch == ']':
        if num ≠ "":
            row.append(int(num))
            num = ""

        if ch == ']':
            if row:
                matrix.append(row)
                row = []

row_sum = []
col_sum = []

for r in matrix:
    row_sum.append(sum(r))

for j in range(len(matrix[0])):
    total = 0
    for i in range(len(matrix)):
        total += matrix[i][j]
    col_sum.append(total)
```

5.

```
print("Row=", row_sum)
print("Col=", col_sum)
```

INPUT

```
[[5,6],[7,8]]
```

OUTPUT

```
Row= [11, 15]
Col= [12, 14]
```

Q6. Maximum Subarray Sum

Score: 6.67 **TC: 2/3 passed**

CODE

```
a = list(map(int, input().split()))
current = a[0]
maximum = a[0]
for i in range(1, len(a)):
    current = max(a[i], current + a[i]
    maximum = max(maximum, current)
print(maximum)
```

INPUT

```
1 2 3 4
```

OUTPUT

```
10
```

6.

Q7. Common Elements Among Three Lists

Score: 10.00 **TC: 3/3 passed**

CODE

```
a, b, c = input().split(';')
a = list(map(int, a.split()))
b = list(map(int, b.split()))
c = list(map(int, c.split()))
result = []
for x in a:
    if x in b and x in c and x not in result:
        result.append(x)
print(result)
```

INPUT

```
1 2 3;2 3 4;2 5
```

7. **OUTPUT**

```
[2]
```

8.

Q8. Matrix Multiplication

Score: 3.33 TC: 1/3 passed

CODE

```
s = input().strip()
a, b = s.split(" B=")
a = a.replace("A=", "").strip()
b = b.strip()
def parse_matrix(x):
    x = x[2:-2]
    rows = x.split("],[")
    matrix = []
    for row in rows:
        row = row.replace("[", "").replace("]", "")
        matrix.append([int(n.strip()) for n in row.split(",")])
    return matrix
A = parse_matrix(a)
B = parse_matrix(b)
result = []
for i in range(len(A)):
    row = []
    for j in range(len(B[0])):
        total = 0
        for k in range(len(B)):
            total += A[i][k] * B[k][j]
        row.append(total)
    result.append(row)
print("[", end="")
for i in range(len(result)):
    print("[", end="")
    print(", ".join(map(str, result[i])), end="")
    print("]", end="")
    if i < len(result) - 1:
        print(",")
print("]")
```

INPUT

```
A=[[1,0],[0,1]] B=[[4,5],[6,7]]
```

OUTPUT

```
[[4, 5],
[6, 7]]
```


Q9. Sort Rotate Min Max

Score: 3.33 **TC: 1/3 passed**

CODE

```
s = input().strip()
s = s.replace("List=", "")
k_pos = s.rfind("k=")
k = int(s[k_pos + 2:].strip())
nums = s[:k_pos].rstrip(",")
arr = list(map(int, nums.split()))
arr.sort()
k = k % len(arr)
rotated = arr[k:] + arr[:k]
print("Sorted=", arr)
print("Rotated=", rotated)
print("Min=", min(arr))
print("Max=", max(arr))
```

INPUT

```
3 12
1
```

OUTPUT

```
Sorted= [1, 3]
Rotated= [1, 3]
Min= 1
Max= 3
```

10.

Q10. Sort Matrix Search

Score: 0.00 TC: 0/3 passed

CODE

```
s = input().strip()
p = s.rfind("Find=")
matrix_part = s[:p].strip()
find = int(s[p + 5:].strip())
start = matrix_part.find("[[")
end = matrix_part.rfind("]]") + 2
m = matrix_part[start:end]
m = m[2:-2]
rows = m.split("],[")
matrix = []
for row in rows:
    row = row.replace("[", "").replace("]", "")
    matrix.append([int(x.strip()) for x in row.split(",")])
values = []
for row in matrix:
    for x in row:
        values.append(x)
values.sort()
cols = len(matrix[0])
sorted_matrix = []
for i in range(0, len(values), cols):
    sorted_matrix.append(values[i:i + cols])
print("Sorted=", sorted_matrix)
found = False
for i in range(len(sorted_matrix)):
    for j in range(len(sorted_matrix[i])):
        if sorted_matrix[i][j] == find:
            print("Position= (", i, ", ", j, ")", sep="")
            found = True
            break
```

```
            break
        if found:
            break
    if not found:
        print("Not Found")
```

INPUT

```
Matrix=[[9,3],[7,1]], Find=7
```

OUTPUT

```
Sorted= [[1, 3], [7, 9]]
Position= (1, 0)
```